L15 - 知识蒸馏

> *"青出于蓝而胜于蓝"*

📌 本节目标

1. 理解知识蒸馏的核心思想：教师模型 → 学生模型
2. 区分黑盒蒸馏与白盒蒸馏
3. **掌握白盒蒸馏的损失函数和温度 T 的作用**
4. 理解 KL 散度的含义
5. 能读懂 MiniMind 的蒸馏实现

📚 前置知识

- L12 预训练（交叉熵损失）
- L13 监督微调 SFT
- 概率分布和信息论基础（Softmax、熵）

---

1. 什么是知识蒸馏?

1.1 直觉理解

想象一位经验丰富的教授（**教师模型**，Teacher）和一位刚入学的学生（**学生模型**，Student）。

传统训练方式：学生只看教科书（标签数据）学习。 知识蒸馏：学生不仅看教科书，还听教授讲课（获取教授的思考过程）。

教授的"思考过程"是什么？就是教师模型对每个 token 输出的**完整概率分布**——不仅告诉你答案是"猫"，还告诉你"狗"的概率是 0.1、"虎"的概率是 0.05.....这些"软信息"包含了丰富的知识。

1.2 为什么需要蒸馏?

问题	说明
大模型太大	GPT-4 级别的模型无法部署到手机或嵌入式设备
推理成本高	大模型的 API 调用成本高
小模型太弱	直接训练小模型效果不够好
蒸馏的价值	让小模型获得接近大模型的效果

---

2. 黑盒蒸馏 vs 白盒蒸馏

2.1 黑盒蒸馏

黑盒蒸馏不需要接触教师模型的内部状态，只使用教师模型的**输出结果**。

```
教师模型（大模型）
    |
    ▼ 生成文本（硬标签）
"量子纠缠是一种量子力学现象，当两个粒子..."
    |
    ▼ 作为训练数据
学生模型（小模型）← 像正常 SFT 一样训练
```

本质上就是：**用大模型生成高质量数据，然后用这些数据训练小模型**。

MiniMind 的黑盒蒸馏：SFT 数据中大量高质量回答来自 **Qwen3、DeepSeek R1** 等强模型。这些数据本身就是黑盒蒸馏的产物。

2.2 白盒蒸馏

白盒蒸馏需要访问教师模型的**内部状态**（logits 分布），不仅学习"正确答案"，还学习教师的"思考方式"。

```
教师模型（大模型）
    |
    ▼ 输出 logits 分布（软标签）
[猫: 0.7, 狗: 0.15, 虎: 0.05, 兔: 0.03, ...]
    |
    ▼ KL 散度损失
学生模型（小模型）← 学习匹配教师的分布
```

2.3 对比总结

维度	黑盒蒸馏	白盒蒸馏
需要教师模型内部状态	否	是（需要 logits）
信息量	少（只有硬标签）	多（完整概率分布）
实现难度	低（生成数据+SFT）	高（需要同时运行两个模型）
适用场景	教师模型只能通过 API 访问	教师模型可以本地运行
效果	好	通常更好

---

3. 硬标签 vs 软标签

3.1 什么是硬标签?

硬标签就是 one-hot 编码——只告诉你正确答案是什么：

```
正确答案: "猫"
硬标签: [0, 0, 1, 0, 0, ...]  ← 只有"猫"的位置是1
```

3.2 什么是软标签?

软标签是模型输出的完整概率分布：

```
教师模型输出:
软标签: [0.01, 0.02, 0.70, 0.15, 0.05, 0.03, ...]
          鸟     鱼     猫     狗     虎     兔
```

3.3 软标签包含更多信息

关键洞察：软标签中"猫和狗的概率接近"这个信息非常有价值——它告诉学生模型，猫和狗在视觉/语义上是相似的，而猫和鱼的差异很大。

硬标签只说"答案是猫"。 **软标签**说"答案最可能是猫，但和狗也有一定相似性，和鱼差很远"。

这些隐含的关系信息被称为 **dark knowledge**（暗知识），是蒸馏的核心价值。

---

4. 温度 T 的作用

4.1 带温度的 Softmax

标准 Softmax：

$$p_i = \frac{\exp(z_i)}{\sum_j \exp(z_j)}$$

带温度的 Softmax：

$$p_i^T = \frac{\exp(z_i / T)}{\sum_j \exp(z_j / T)}$$

其中 T 是温度参数。

4.2 温度对分布的影响

假设 logits 为 [5.0, 2.0, 1.0]：

温度 T	Softmax 分布	特点
0.5	[0.993, 0.006, 0.001]	更"尖锐"，集中在最大值
1.0	[0.844, 0.114, 0.042]	标准分布
2.0	[0.656, 0.220, 0.124]	更"平缓"
5.0	[0.465, 0.284, 0.251]	非常"平缓"，接近均匀
→∞	[0.333, 0.333, 0.333]	完全均匀分布

4.3 为什么蒸馏需要高温度？

当 $T = 1$ 时，教师模型的输出分布通常非常"尖锐"——正确答案的概率远高于其他选项，接近硬标签。此时软标签携带的额外信息有限。

提高温度 T，分布变得更"平缓"： - 不同类别之间的概率差异被放大 - "暗知识"得以显现 - 学生能学到更多类别间的关系

常用温度：T = 2~5。MiniMind 通常使用 T = 2。

4.4 温度的直觉类比

想象一位教授批改试卷： - **T=1（低温）**："A 选项对，BCD 都错。"——信息量少 - **T=3（高温）**："A 选项最对（70分），B 也有道理（20分），C 部分正确（8分），D 完全不对（2分）。"——信息量丰富

---

5. 白盒蒸馏的损失函数

5.1 完整损失公式

白盒蒸馏的损失函数由两部分组成：

$\mathcal{L} = \alpha \cdot \mathcal{L}_{\text{CE}} + (1 - \alpha) \cdot T^2 \cdot \mathcal{L}_{\text{KL}}$

其中： - $\mathcal{L}_{\text{CE}}$： **硬标签损失**——学生预测 vs 真实标签的交叉熵 - $\mathcal{L}_{\text{KL}}$： **软标签损失**——学生分布 vs 教师分布的 KL 散度 - α：平衡系数（通常 0.1~0.5） - T：温度 - T^2：补偿因子（下面解释）

5.2 硬标签损失

$\mathcal{L}_{\text{CE}} = -\sum_i y_i \log p_i^S$

这就是标准的交叉熵损失，和正常训练一样。

5.3 软标签损失（KL 散度）

$\mathcal{L}_{\text{KL}} = \text{KL}(p_{\text{teacher}}^T \| p_{\text{student}}^T) = \sum_i p_{\text{teacher},i}^T \log \frac{p_{\text{teacher},i}^T}{p_{\text{student},i}^T}$

其中 p^T 表示使用温度 T 的 Softmax 输出。

5.4 为什么要乘以 T²?

使用温度 T 的 Softmax 后，梯度会被缩小 $1/T^2$ 倍（因为 logits 除以了 T，反向传播时梯度也除以 T，两次导数乘积为 $1/T^2$）。乘以 T^2 是为了**补偿这个缩放**，使硬标签损失和软标签损失的梯度量级一致。

5.5 α 的选择

- α 接近 1：更依赖硬标签（真实标签），蒸馏效果减弱
- α 接近 0：更依赖软标签（教师分布），可能不稳定
- 常用值：α = 0.1~0.5

---

6. KL 散度详解

6.1 什么是 KL 散度?

KL 散度（Kullback-Leibler Divergence）衡量两个概率分布之间的"距离"：

$$\mathrm{KL}(P||Q) = \sum_i P(i)\log\frac{P(i)}{Q(i)}$$

直觉理解：如果用分布 Q 来编码来自分布 P 的信息，**多消耗的比特数**。

6.2 KL 散度的性质

1. **非负性**：$\mathrm{KL}(P||Q) \geq 0$，当且仅当 P = Q 时等于 0
2. **非对称性**：$\mathrm{KL}(P||Q) \neq \mathrm{KL}(Q||P)$
3. 不是真正的"距离"（不满足对称性和三角不等式）

6.3 在蒸馏中的角色

蒸馏中我们希望学生的分布 p_S^T 尽可能接近教师的分布 p_T^T：

$$\min \mathrm{KL}(p_T^T||p_S^T)$$

当学生完美匹配教师时，KL 散度为 0。

6.4 KL 散度 vs 交叉熵

它们的关系：

$$\mathrm{KL}(P||Q) = H(P,Q) - H(P)$$

其中 $H(P,Q)$ 是交叉熵，$H(P)$ 是 P 的自信息熵。由于 $H(P)$ 是常数（教师分布固定），最小化 KL 散度等价于最小化交叉熵。

---

7. 代码实现

7.1 白盒蒸馏的 PyTorch 实现

```
import torch
import torch.nn.functional as F

def distillation_loss(student_logits, teacher_logits, labels, T=2.0, alpha=0.3):
    """
    白盒知识蒸馏损失函数

    Args:
        student_logits: 学生模型输出 [batch, seq_len, vocab_size]
        teacher_logits: 教师模型输出 [batch, seq_len, vocab_size]
        labels: 真实标签 [batch, seq_len]
```

```
        T: 温度
        alpha: 硬标签权重
    """
    # 1. 硬标签损失（标准交叉熵）
    hard_loss = F.cross_entropy(
        student_logits.view(-1, student_logits.size(-1)),
        labels.view(-1),
        ignore_index=-100
    )

    # 2. 软标签损失（KL 散度）
    student_soft = F.log_softmax(student_logits / T, dim=-1)
    teacher_soft = F.softmax(teacher_logits / T, dim=-1)
    soft_loss = F.kl_div(
        student_soft.view(-1, student_soft.size(-1)),
        teacher_soft.view(-1, teacher_soft.size(-1)),
        reduction='batchmean'
    )

    # 3. 总损失
    loss = alpha * hard_loss + (1 - alpha) * T * T * soft_loss
    return loss
```

7.2 蒸馏训练循环

```
# 伪代码展示蒸馏训练流程
teacher_model.eval()  # 教师模型固定，不训练

for batch in dataloader:
    input_ids, labels = batch

    # 教师模型前向（不计算梯度）
    with torch.no_grad():
        teacher_logits = teacher_model(input_ids)

    # 学生模型前向
    student_logits = student_model(input_ids)

    # 计算蒸馏损失
    loss = distillation_loss(student_logits, teacher_logits, labels, T=2.0, alpha=0.3)

    # 反向传播（只更新学生模型）
    loss.backward()
    optimizer.step()
    optimizer.zero_grad()
```

---

8. MiniMind 的蒸馏实现

8.1 关键文件

- `trainer/train_distillation.py`：白盒蒸馏训练脚本

8.2 MiniMind 的双重蒸馏策略

MiniMind 同时使用了两种蒸馏：

1. 黑盒蒸馏（隐式） SFT 训练数据中的高质量回答来自 Qwen3、DeepSeek R1 等强模型。用这些数据做 SFT，本质上就是黑盒蒸馏。

2. 白盒蒸馏（显式） `train_distillation.py` 实现了完整的白盒蒸馏，使用教师模型的 logits 分布来指导学生模型训练。

8.3 蒸馏训练配置

MiniMind 蒸馏训练的典型配置：

```
# 蒸馏超参数
temperature = 2.0        # 温度
alpha = 0.3              # 硬标签权重
teacher_model = "..."    # 教师模型路径
student_model = "..."    # 学生模型路径
```

8.4 显存管理

白盒蒸馏需要同时加载教师和学生两个模型，显存压力更大。MiniMind 的处理方式：

```
# 教师模型设为 eval 模式，不计算梯度
teacher_model.eval()
for param in teacher_model.parameters():
    param.requires_grad = False

# 教师模型可以用半精度节省显存
teacher_model.half()
```

---

9. 蒸馏的实际效果

9.1 效果对比

一般来说，蒸馏后的学生模型表现：

```
教师模型（大） > 蒸馏学生（小） > 独立训练学生（小）
```

蒸馏不能让学生超越教师（受限于模型容量），但能显著缩小与教师的差距。

9.2 为什么蒸馏有效？

1. **软标签提供更丰富的监督信号**：每个训练样本的信息量更大
2. **暗知识传递**：类别间的关系信息帮助学生更好地理解任务
3. **正则化效果**：软标签比硬标签更平滑，起到隐式正则化的作用
4. **更高效的知识传输**：教师已经从大量数据中提炼了知识，学生直接学习这些"精华"

9.3 蒸馏的局限性

1. **受限于教师质量**：教师不好，学生也好不了
2. **容量瓶颈**：学生太小时，无法完全吸收教师的知识
3. **计算开销**：白盒蒸馏需要同时运行两个模型
4. **数据依赖**：仍需要足够的训练数据

🎤 面试考点

Q1: 知识蒸馏的原理是什么？

答：知识蒸馏通过一个大的教师模型来指导小的学生模型训练。核心思想是让学生模型不仅学习硬标签（正确答案），还学习教师模型的软标签（完整概率分布）。软标签包含丰富的"暗知识"——类别间的相似性关系，这些信息能帮助学生更好地学习。

Q2: 温度 T 的作用是什么？

答：温度 T 用于软化教师模型的输出分布。当 T=1 时，分布通常很"尖锐"，接近硬标签；提高 T 使分布更"平缓"，让不同类别间的概率差异更明显，"暗知识"得以显现。常用 T=2~5。同时，蒸馏损失要乘以 T^2 来补偿温度对梯度的缩放影响。

Q3: 黑盒蒸馏 vs 白盒蒸馏的区别？

答： - **黑盒蒸馏**：只使用教师模型的输出文本（硬标签），不需要访问模型内部。本质是用大模型生成数据来训练小模型。优点是实现简单，教师可以是 API；缺点是信息量有限。 - **白盒蒸馏**：使用教师模型的 logits 分布（软标签），需要访问模型内部。通过 KL 散度让学生匹配教师的分布。优点是信息量更丰富；缺点是需要同时运行两个模型。

Q4: KL 散度是什么？它在蒸馏中的作用？

答：KL 散度衡量两个概率分布之间的"距离"：$\mathrm{KL}(P||Q) = \sum_i P(i)\log\frac{P(i)}{Q(i)}$。它具有非负性和非对称性。在蒸馏中，用 KL 散度来最小化学生分布和教师分布之间的差异，让学生模型学会匹配教师的"思考方式"。

Q5: 蒸馏损失为什么要乘以 T^2?

答：使用温度 T 后，logits 被除以 T，反向传播时梯度也被缩小了。具体地，softmax 的梯度与 $1/T$ 成正比，KL 散度中有两次 softmax 操作的梯度，因此整体梯度被缩小了约 $1/T^2$。乘以 T^2 是为了补偿这个缩放，使软标签损失和硬标签损失的梯度量级一致。

Q6: 蒸馏后的学生模型能超越教师吗?

答：在同一任务上，蒸馏后的学生一般不会超越教师，因为受限于模型容量。但在某些特定场景下可能例外：(1) 教师模型过拟合了，学生通过蒸馏反而获得了更好的泛化；(2) 学生在特定子任务上通过蒸馏获得了更集中的能力。

✅ 自测题

1. 手动计算：给定 logits [3.0, 1.0, 0.5]，分别计算 T=1 和 T=3 时的 Softmax 分布。
2. 写出白盒蒸馏的完整损失函数，解释每个符号的含义。
3. 为什么说软标签比硬标签包含更多信息？举一个具体的例子。
4. MiniMind 如何同时使用黑盒蒸馏和白盒蒸馏?
5. 如果教师模型本身质量很差，蒸馏会有什么后果?

⏭️ 下一节预告

L16 - 完整模型组装：我们已经学完了所有训练技术。最后一节，让我们回顾所有组件，像搭乐高一样把它们组装成完整的 MiniMind 模型，计算参数量，理解数据流动的全过程。